Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

# Information Systems Integration

## Part 1 – Background

### Integration problems, approaches and types

# Background

"Interesting applications rarely live in isolation" [1]



# Background

- Distributed applications vs Integration [1]
  - Tightly coupled and synchronous vs loosely coupled asynch
  - Dependent applications vs independent applications

# Background

- Integration challenges [1]
  - Networks are unreliable (delays or interruptions)
  - Networks are slow
  - Applications are different (OS, Prog. Languages, HW, etc.)
  - Change is inevitable (“loosely coupling is beautiful”)



# Background

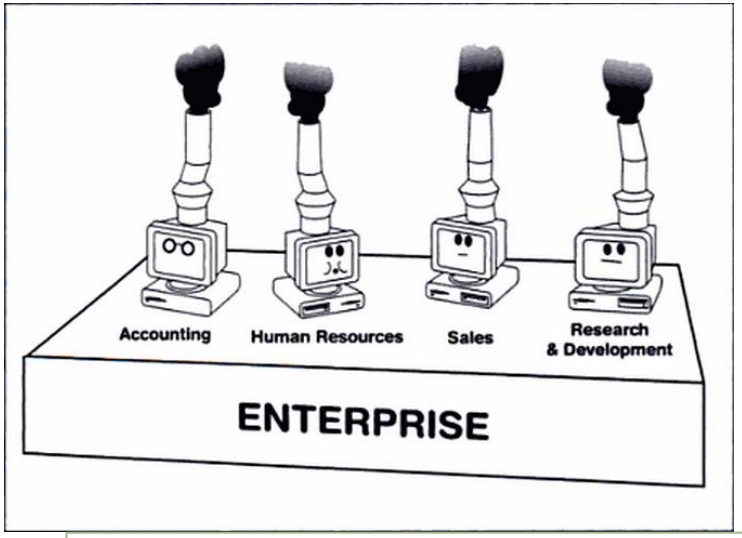
- Remember common integration mechanisms [1]
  - File transfer (location, format, structure, encoding, etc.)
    - Example: see “View Catalog” from previous lecture (download by ftp)
  - Shared database
    - Example: see “Checking status” from previous lecture
  - Remote procedure call - RPC (synchronous)
  - Messaging (asynchronous)

Both with advantages and disadvantages

# Background

## The need for integration [1]

- Companies are comprised of dozens of applications
- Applications were acquired in an evolutionary way…




[David Linthicum, "Enterprise Application Integration", Addison–Wesley]

It is a mess. Shouldn't CEO be fired? ☺

?

# Background

The need for integration [1]

- Developing “one fit all” application is impossible…
  - ERPs are an approximate, but far away from “one fit all”
- Multiple applications bring flexibility (choosing best one)
- When customers, partners and internal users interact with business they must not be aware of the application’s boundaries: **unification perspective**



Applications Integration!

# Background

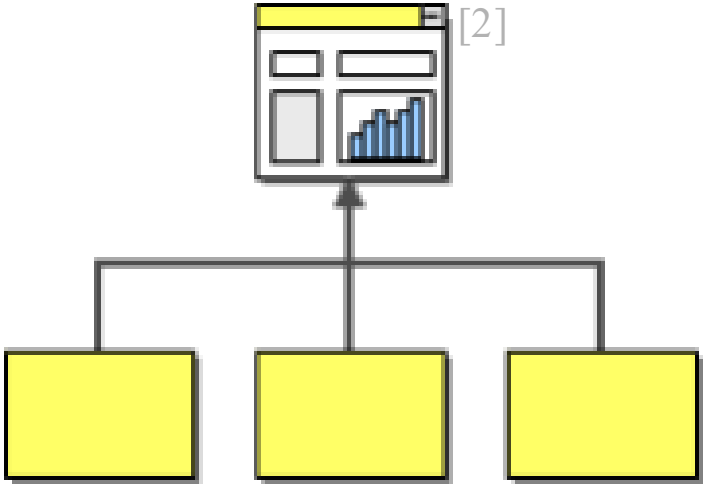
- Integration is a hard topic [1]
  - There is no books to cover all the spectrum of integration
    - Focus on technologies and tools and not specific problems
    - Most knowledge comes from past integrations problems
      - Six **types of problems** (market needs):
        1. Information Portals / Dashboards
        2. Data replication
        3. Shared business functions
        4. Service oriented architectures
        5. Distributed business processes
        6. Business-to-business integration

# Background

Integration is a hard topic [1]

1. The information **portal** need

Joins information from different sources


[2]

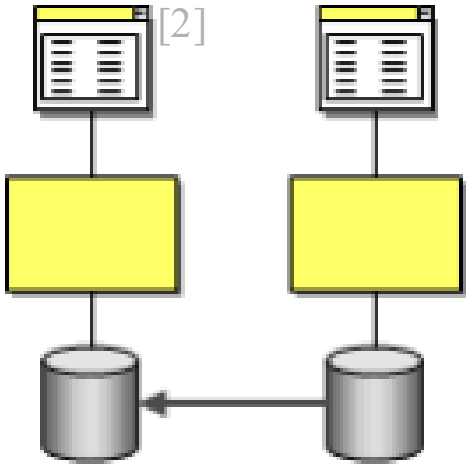
User sees one whole system

# Background

Integration is a hard topic [1]

2. The **data replication** need

Some data is common to several departments (e.g., customer)



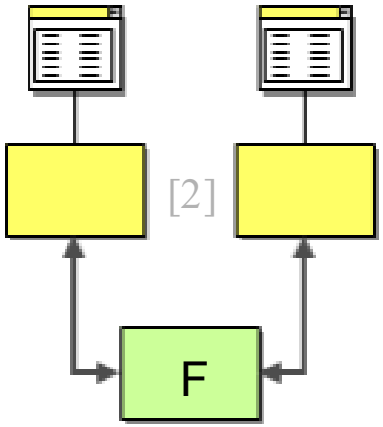

Replication means **source reading**, apply **formats**, data **transformations**, etc., and finally **data storage**

# Background

Integration is a hard topic [1]

3. **Shared business function** need

E.g., verify validity of social-security number in a unique function (eliminate replications!)




Internally only perspective…
This approach is easily replaceable by next approach

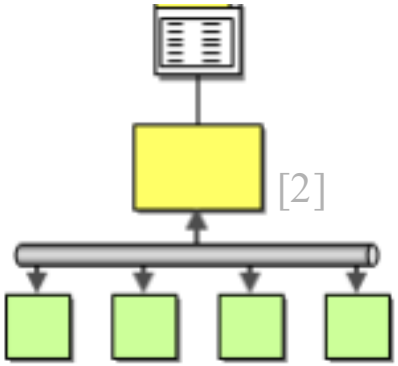
In company internal shared functions focus

# Background

Integration is a hard topic [1]

4. **Service-oriented architecture** need

Collection of “shared functions” (services) as base model


[2]

(Lots of shared functions)

Service discovery and negotiation needed!
**(directory service and interfaces)**

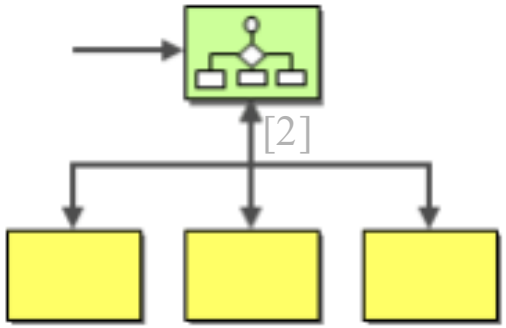
In company (and outside) shared functions/services

# Background

Integration is a hard topic [1]

**5. Distributed business processes** need

Several operations spread across various systems…




'Transactions' needed.
**Functions which call (shared) functions in a consistent way**

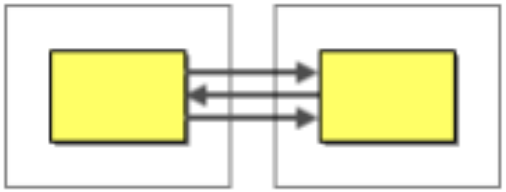
Inter departments focus

# Background

Integration is a hard topic [1]

6. **Business-to-business integration** need

Communication with external systems/companies…



[2]

Standardized data formats are critical!
**(Retail is a good example)**

Inter company focus

# Integration approaches

# Background

**Integration approaches**

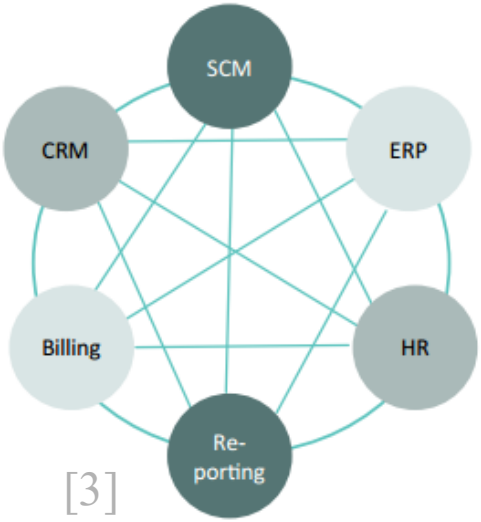
1. Point-to-point integration
2. Integration through middleware

# Background

## Integration approaches

### 1. Point-to-point integration

A Link connects each pair of applications that must communicate [**N(N-1)/2 links**!]




[3]

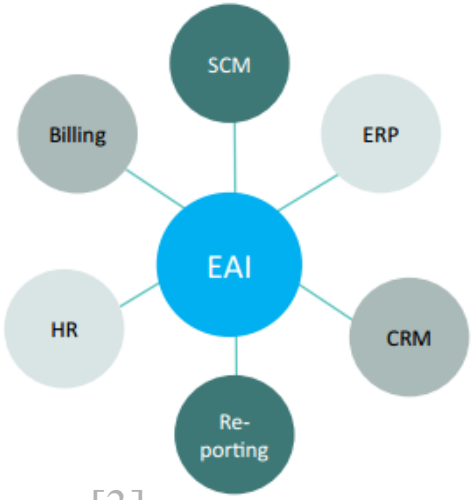
For few applications scenarios!
Maintenance!
Growing complexity!

# Background

Integration approaches

2. Integration through middleware

Common layer which export interface for each functionality




[3]

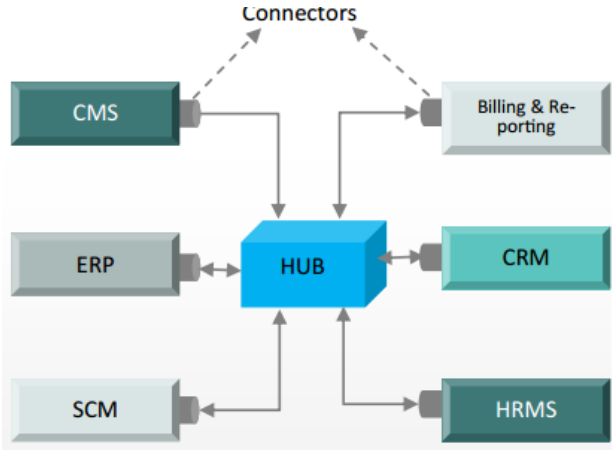
Eases maintenance and growing complexity;
Middleware could be loosely or tightly coupled;
But single point of failure (centralized!) and middleware maintenance

# Background

Integration approaches

2. Integration through middleware (example)

2.1 Message Hubs - Message exchange without peer knowledge needed




[3]

Example: Apache MQTT

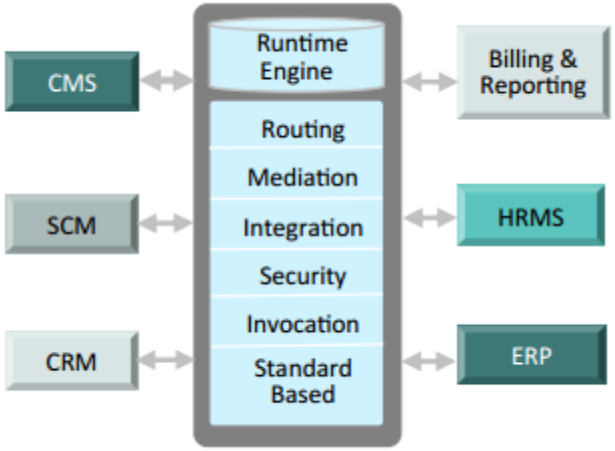
Hub service reusing!
Scalable
Loosely coupled
**SPOF**
**Bottleneck**
**Needs adapters/connectors**

# Background

Integration approaches

2. Integration through middleware (example)

2.2. Service Bus - Distributed/P2P Message Hub system like




[3]

No SPOF;
No bottleneck;
Standards;
Converts formats to the end node;
Location, rooting, etc. auto/;
Very scalable;
Provides many adapters

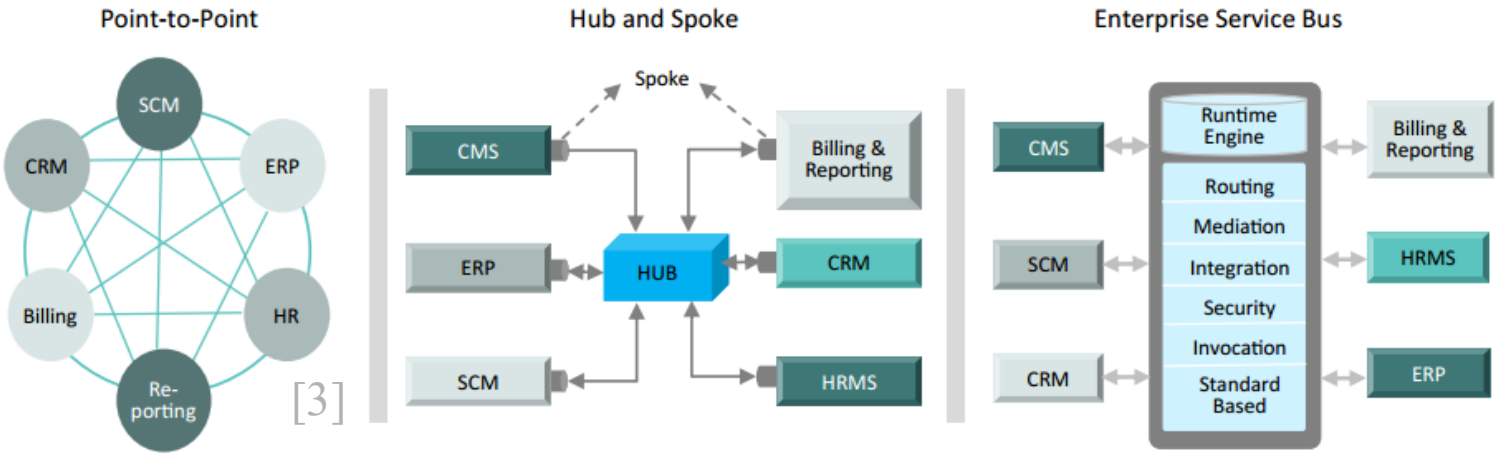
Example: Apache Camel; Microsoft BizTalk; Guaraná DSL, Red Hat Fuse, etc…

# Background

Integration approaches

2. Integration through middleware

Which approach should we choose?




Synch
High effort

Developers

Asynch
(Ready to be used)

# Background

Integration approaches

2. Integration through middleware

Which approach should I choose?

- How many applications do I need to integrate?
- Will I need to add additional applications in the future?
- How many communication protocols will I need to use?
- Do my integration needs include routing, forking, or aggregation?
- How important is scalability to my organization?
- Does my integration situation require asynchronous messaging, publish / consume messaging models, or other complex multi-application messaging scenarios?

[3]

# Integration Types

# Background

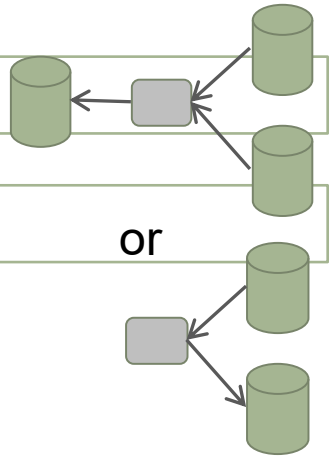
**Integration types**

- Data level integration
- Application (API) level integration
- Method level integration
- User interface level integration

# Background

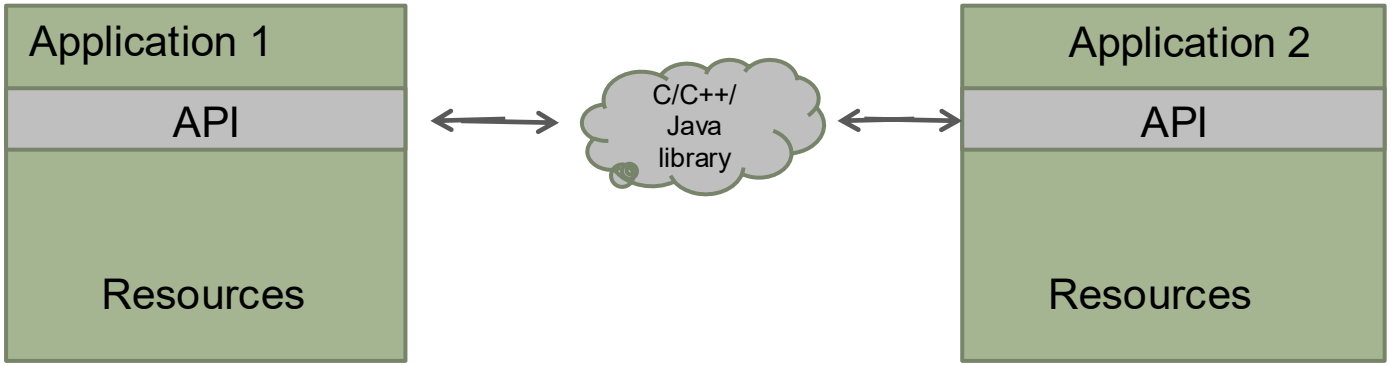
Integration types

- **Data level integration**
  - Data is extracted from one database, it is processed and stored in another database
  - Applications are not modified! **No testing or deploy needed!**
  - Probably, no application's source-code…
  - In-the-middle replicator software



# Background

- Integration types
  - **Application (API) level integration**
    - Relies on application's (well known) interfaces
    - Example: Java application server hosting application and exposing Web Service/RESTful service



# Background

- Integration types
  - **Method level integration**
    - Relies on some form of RPC
    - Example: legacy software offering RPC interaction
      - App 1: RPC stub <-----> App 2: RPC Server

# Background

- Integration types
  - **User interface level integration**
    - More primitive but…
    - “Export display” like approach (UNIX)
    - Key loggers, GUI robots, etc…

# References/Bibliography

[1] Gregor Hohpe, and Bobby Woolf, “Enterprise Integration Patterns: Designing Building, and Deploying Messaging Solutions”, Pearson Education, Inc., 2003

[2] Mathias Weske, “Business Process Management: Concepts, Languages, Architectures”, Springer Berlin Heidelberg, 2nd ed. 2012.

[3] Anjali Gupta, “Enterprise Application Integration”, White Paper, SPAN Chandigarh, 2014.